

Reporte de Práctica: Conexión SQLite con Pandas

Instituto Tecnológico Superior de Ciudad Constitución

Ingeniería en Sistemas Computacionales

1. Descripción General

La práctica consiste en cargar el archivo *members.csv* mediante la librería Pandas, establecer una conexión a una base de datos SQLite llamada *database_iniciales.db* e insertar los datos en una tabla. Posteriormente se analiza el esquema generado y se responden preguntas sobre el tipo de dato de *birth_date* y las ventajas de Pandas en el proceso.

2. Tecnologías Utilizadas

Se utilizaron las siguientes herramientas y librerías en un entorno de Python 3 con ambiente virtual:

Librería / Herramienta	Versión	Uso
Python	3.11+	Lenguaje base
pandas	2.x	Carga y escritura de datos
sqlite3	stdlib	Conexión y operaciones en SQLite
Jupyter Notebook	6.x / 7.x	Entorno interactivo de desarrollo

3. Descripción del Código

3.1 Importación de librerías

Se importan *pandas* como *pd* y el módulo estándar *sqlite3*. No se requieren instalaciones adicionales ya que *sqlite3* forma parte de la biblioteca estándar de Python.

```
import pandas as pd
import sqlite3
```

3.2 Carga del archivo CSV

Se utiliza *pd.read_csv()* para leer el archivo *members.csv*. El resultado es un DataFrame con 5,000 filas y 10 columnas. Pandas infiere automáticamente el tipo de dato de cada columna durante la lectura.

```
df = pd.read_csv('members.csv')
```

3.3 Creación de la conexión a SQLite

La función *sqlite3.connect()* crea el archivo *database_iniciales.db* en el directorio de trabajo si no existe, o lo abre si ya fue creado previamente. La variable *conn* representa el objeto de conexión

activo.

```
conn = sqlite3.connect('database_iniciales.db')
```

3.4 Escritura del DataFrame en SQLite

El método `to_sql()` de Pandas escribe el contenido del DataFrame en la tabla *members* dentro de la base de datos. El parámetro `if_exists='replace'` indica que si la tabla ya existe debe eliminarse y recrearse. El parámetro `index=False` evita que el índice del DataFrame se almacene como columna adicional.

```
df.to_sql('members', conn, if_exists='replace', index=False)
```

3.5 Consulta del esquema SQL

Para obtener la definición de la tabla se consulta el catálogo interno de SQLite *sqlite_master*. Este contiene la sentencia *CREATE TABLE* que fue generada automáticamente por Pandas.

```
cursor = conn.cursor()
cursor.execute("SELECT sql FROM sqlite_master WHERE type='table' AND name='members';")
schema = cursor.fetchone()[0]
print(schema)
```

3.6 Esquema resultante

```
CREATE TABLE "members" (
  "participant_id" TEXT,
  "first_name" TEXT,
  "last_name" TEXT,
  "birth_date" TEXT,
  "address" TEXT,
  "phone_number" TEXT,
  "country" TEXT,
  "institute" TEXT,
  "occupation" TEXT,
  "register_time" INTEGER
)
```

4. Análisis del Esquema

4a. ¿Por qué *birth_date* arroja tipo TEXT?

Al examinar los valores del campo *birth_date* en el CSV se encuentran al menos dos formatos distintos mezclados en la misma columna:

Valor en CSV	Formato observado
05-feb-91	DD-MON-YY (abreviatura en minúsculas)
11 Jan 1993	DD Mon YYYY (nombre en inglés, espacio como separador)
31-jul-93	DD-MON-YY (abreviatura en minúsculas)
22 Jan 2003	DD Mon YYYY

Cuando Pandas detecta una columna cuyos valores no siguen un patrón de fecha uniforme y reconocible, no puede inferir automáticamente el tipo *datetime64*. Por lo tanto, la columna queda como tipo *object* (cadena de texto en Python). Al transferir ese tipo a SQLite mediante `to_sql()`, la librería lo

mapea al tipo de afinidad *TEXT*.

Para convertir correctamente esta columna se requiere utilizar explícitamente:

```
df['birth_date'] = pd.to_datetime(df['birth_date'], format='mixed')
```

4b. Principal ventaja de Pandas en este proceso

La ventaja principal es la automatización completa del esquema de base de datos. Sin Pandas, el desarrollador tendría que:

1. Analizar manualmente las columnas del CSV y determinar sus tipos.
2. Escribir la sentencia CREATE TABLE con los tipos SQLite correspondientes.
3. Iterar fila por fila ejecutando sentencias INSERT INTO para cada registro.

Con el método *to_sql()*, todo ese proceso ocurre en una sola línea de código. Pandas infiere los tipos, genera la definición de la tabla e inserta los 5,000 registros en lotes internos de forma eficiente. Esto reduce el tiempo de desarrollo, minimiza la posibilidad de errores de tipo y hace el código más mantenible y portable.

5. Conclusión

La práctica demuestra cómo Pandas actúa como puente eficiente entre archivos planos y bases de datos relacionales. La integración con SQLite mediante *to_sql()* y *read_sql_query()* simplifica significativamente el flujo de trabajo de carga, almacenamiento y consulta de datos. El análisis del esquema evidencia la importancia de mantener formatos de fecha consistentes en los datos fuente para garantizar una tipificación correcta.